

Documentação do Primeiro MVP

Torque Gestão — Sistema de Gestão Automotiva

Nome do projeto	Torque Gestão — Sistema de Gestão Automotiva
Integrantes	Miguel Angelo Dufloth Filho; Miguel Angel Balladares Huertas; Leonardo Lotério de Lima; Lucas Honorato dos Santos
Repositório	https://github.com/miguetttohuertas/torque-gestao
Protótipo navegável	https://torque-gestao.surge.sh
Data de elaboração	Joinville, 17 de agosto de 2026
Última revisão	Joinville, 18 de agosto de 2026
Entrega desta documentação	27 de agosto de 2026, até 23h59

Sumário

1	Status Atual do Projeto	2
1.1	Entregas concluídas	2
1.2	O que ainda não foi iniciado	2
1.3	Síntese	3
2	Próximo MVP (MVP 2)	3
2.1	Objetivo	3
2.2	Escopo funcional priorizado	3
2.3	Arquitetura e modelo de dados	4
2.4	Entregáveis técnicos	5
2.5	Critérios de aceite (Definition of Done do MVP 2)	5
2.6	Cronograma e divisão de responsabilidades	6
2.7	Acompanhamento e comunicação	7

2.8 Riscos específicos do MVP 2	7
---	---

1 Status Atual do Projeto

Este capítulo descreve o estado consolidado do projeto Torque Gestão ao final da migração do material herdado do PAC V e do onboarding inicial da equipe no PAC VI, servindo de linha de base (*baseline*) para o planejamento do próximo MVP.

1.1 Entregas concluídas

- **Protótipo de alta fidelidade:** 19 telas mapeadas e navegáveis, desenvolvidas em React (Babel Standalone), publicadas em <https://torque-gestao.surge.sh>. Cobrem os três perfis de usuário do sistema — Administrador, Mecânico e Cliente — incluindo login unificado, dashboards, gestão completa de ordens de serviço (OS), cadastro de clientes e veículos, catálogo de serviços/peças, gestão de usuários, configurações e portal do cliente.
- **Documentação de especificação (herdada do PAC V):** relatório final com Introdução, Público-alvo, Objetivos, Definição das 19 interfaces, Mapeamento de Riscos (incluindo matriz de riscos), 6 Requisitos Funcionais (RF01–RF06), 6 Requisitos Não Funcionais (RNF01–RNF06) e Stack Tecnológica — disponível em `docs/academic/report.pdf`.
- **Diagramas técnicos:** C4 (Contexto, Containers, Componentes), fluxo de processo de abertura de OS, diagrama de sequência, diagrama de estados da OS, modelo Entidade-Relacionamento e diagrama de casos de uso — todos versionados em `docs/diagramas/` (Mermaid e PlantUML, com exportações PNG).
- **Design system:** biblioteca de componentes de UI documentada em `docs/design-system/` (botões, cards, badges, cores, tipografia, inputs, navegação, tabelas, tokens de espaçamento) e prévia consolidada em `packages/ui/components`.
- **Repositório do PAC VI estruturado:** novo repositório criado (`torque-gestao`), com monorepo organizado em `apps/`, `packages/`, `docs/`, `assets/` e `scripts/`, e pipeline de automação (`scripts/compile_report.py`, `generate_diagrams.py`, `generate_docs.py`) para manter documentação e diagramas sincronizados com o protótipo.
- **Equipe completa como colaboradora do repositório:** os quatro integrantes já têm acesso de escrita ao repositório do PAC VI, superando a pendência registrada no diagnóstico inicial da disciplina.

1.2 O que ainda não foi iniciado

- **Back-end:** nenhuma linha de código de servidor foi escrita. Não existe ainda o pacote `apps/api` nem qualquer endpoint FastAPI.

- **Banco de dados:** o PostgreSQL definido na stack ainda não foi provisionado; os dados do protótipo são inteiramente mockados em `apps/prototype/src/mock-data.jsx`, sem persistência real.
- **Autenticação real:** o login do protótipo simula o acesso por perfil via `localStorage`, sem validação de credenciais, emissão de token ou controle de acesso efetivo (RF06 ainda não implementado).
- **Integração contínua e deploy:** Docker, GitHub Actions, e as hospedagens em Render e Vercel previstas na stack ainda não foram configuradas.

1.3 Síntese

Em resumo, o MVP atual do projeto é um **protótipo de interface completo e navegável, com toda a especificação funcional, os riscos e a arquitetura já documentados**, mas sem nenhuma camada de back-end, persistência ou autenticação real implementada. O próximo MVP, detalhado na seção seguinte, tem como missão transformar esse protótipo estático em uma aplicação funcional de ponta a ponta.

2 Próximo MVP (MVP 2)

2.1 Objetivo

Evoluir o Torque Gestão de um protótipo de interface com dados mockados para uma **aplicação funcional de ponta a ponta**, com back-end real em FastAPI, persistência em PostgreSQL e autenticação efetiva, cobrindo o fluxo essencial de uma oficina: cadastro de clientes e veículos, abertura e acompanhamento de ordens de serviço, consulta ao catálogo de serviços e peças, e leitura do status real pelo próprio cliente.

2.2 Escopo funcional priorizado

O escopo do MVP 2 foi priorizado a partir dos Requisitos Funcionais já especificados no relatório do PAC V. Os requisitos ligados ao fluxo central da oficina (RF01 a RF06) compõem o núcleo do MVP; funcionalidades administrativas de apoio (relatórios financeiros e KPIs avançados, gestão completa de usuários pela interface) ficam para uma terceira iteração, priorizando primeiro um fluxo de ponta a ponta funcionando de verdade.

Tabela 1: Escopo funcional do MVP 2

ID	Requisito	Escopo no MVP 2
RF06	Autenticação e Controle de Acesso	Endpoint de login real, emissão de JWT e middleware de RBAC (Admin, Mecânico, Cliente) protegendo as demais rotas. Implementado primeiro por ser pré-requisito de todo o restante.
RF01	Gerenciamento de Clientes	CRUD completo de clientes via API, com validação de CPF/CNPJ e busca, substituindo o mock em pelo menos duas telas do protótipo.
RF02	Cadastro de Veículos	CRUD de veículos vinculados a um cliente, com validação de placa (padrão Mercosul e antigo).
RF03	Emissão de Ordens de Serviço	Criação de OS com itens de mão de obra e peças separados, cálculo automático do orçamento a partir do catálogo.
RF04	Acompanhamento de Status	Persistência do fluxo de estados da OS (Aguardando Diagnóstico → Em Execução → Aguardando Peças → Finalizada → Entregue), com registro em <code>HISTORICO_STATUS</code> a cada transição.
RF05	Catálogo de Serviços e Peças	Endpoints de consulta (e cadastro por Admin) do catálogo, usados na composição do orçamento da OS.

Portal do cliente: passa a ler dados reais no MVP 2. Reaproveitando os mesmos endpoints de RF01, RF02 e RF04 (sem back-end adicional), o cliente autenticado via RF06 consulta o status real de suas OS e o histórico real de seus veículos — eliminando a lacuna que existia no rascunho anterior deste documento, em que o portal ficava ambíguo entre incluído e não incluído no MVP 2.

Fora do escopo do MVP 2 (reservado para o MVP 3): relatório financeiro e KPIs avançados do dashboard (dados agregados reais), tela de configurações do sistema, e refinamentos de UX do portal do cliente que dependem de infraestrutura adicional — notificações em tempo real (e-mail/SMS) e atualização automática via *WebSocket/polling*, que exigem componentes fora do escopo de back-end definido nesta iteração.

2.3 Arquitetura e modelo de dados

- **Novo pacote `apps/api`:** serviço FastAPI organizado em camadas de *routers* (endpoints REST), *schemas* (validação Pydantic) e *models* (ORM SQLAlchemy), seguindo a stack já definida no relatório do PAC V.
- **Banco de dados:** PostgreSQL, com schema derivado diretamente do modelo Entidade-

Relacionamento já modelado em `docs/diagramas/modelo-er.md` — entidades `CLIENTE`, `VEICULO`, `USUARIO`, `ORDEM_SERVICO`, `ITEM_OS`, `HISTORICO_STATUS`, `CATALOGO_SERVICO` e `CATALOGO_PECA` — versionado com migrations (Alembic).

- **Autenticação:** tokens JWT com claim de perfil (`role`), validados por *dependency injection* do FastAPI em cada rota protegida, implementando o RBAC previsto em RF06/RNF01.
- **Documentação de API:** geração automática de especificação OpenAPI/Swagger (RNF06), publicada em `/docs` do serviço, servindo como coleção de referência para o time de front-end e como contrato formal entre back-end e front-end (ver mitigação do risco de inconsistência de contrato, Seção 2.8).
- **Front-end:** as telas prioritárias do protótipo (login, dashboard admin, lista/nova OS, detalhes da OS, cadastro de clientes e veículos, e as três telas do portal do cliente) passam a consumir a API real via `fetch`, substituindo progressivamente `apps/prototype/src/mock-data.jsx`.

2.4 Entregáveis técnicos

- Pacote `apps/api` com os endpoints de RF01, RF02, RF03, RF04, RF05 e RF06 implementados e testados.
- Script de migrations (Alembic) e *seed* inicial do catálogo de serviços/peças.
- `Dockerfile` do back-end e `docker-compose.yml` (API + PostgreSQL) para ambiente de desenvolvimento local, conforme a stack de containerização já definida.
- Workflow de GitHub Actions executando lint e suíte de testes automatizados (`pytest`) a cada *pull request*.
- Suíte de testes cobrindo as regras de negócio críticas: cálculo de orçamento (RF03) e transições de status da OS (RF04).
- Telas do protótipo (login, gestão de OS, cadastro de clientes/veículos e portal do cliente) conectadas à API real.
- Atualização do README com instruções de execução local (*passo a passo*) e, caso o schema real divirja do modelo mockado, atualização de `docs/diagramas/modelo-er.md`.

2.5 Critérios de aceite (Definition of Done do MVP 2)

O MVP 2 será considerado concluído quando, cumulativamente:

1. o ambiente completo (API + banco de dados) subir localmente com um único comando (`docker compose up`);
2. o login autenticar de fato contra o banco de dados, emitindo um JWT válido e respeitando o RBAC por perfil;

3. cadastro de clientes e veículos persistir no PostgreSQL, sobrevivendo a um *reload* da aplicação;
4. a criação e a atualização de status de uma OS persistirem e respeitarem o fluxo de estados definido em RF04;
5. o catálogo de serviços e peças for consultável via API e utilizado no cálculo do orçamento da OS;
6. a documentação OpenAPI/Swagger estiver publicada e acessível;
7. as telas de login, gestão de OS, cadastro de clientes/veículos **e o portal do cliente (leitura de status e histórico)** consumirem dados reais da API, em vez do mock;
8. o pipeline de CI (lint + testes) estiver executando automaticamente a cada *pull request*.

2.6 Cronograma e divisão de responsabilidades

As datas abaixo são o compromisso interno da equipe a partir de **18 de agosto de 2026**, com as fases distribuídas de forma equilibrada entre os quatro integrantes. Cada responsável principal conduz sua fase e revisa o trabalho das demais via *pull request*; a fase seguinte só começa oficialmente após a anterior atingir sua entrega, mas a Fase 1 corre em paralelo à finalização desta documentação para que já exista progresso técnico concreto até 27/08.

Tabela 2: Cronograma e responsáveis do MVP 2

Fase	Responsável	Período	Entrega da fase
Fase 0 — Documentação e planejamento	Todos os integrantes	18/08 a 27/08	Esta documentação revisada (escopo, cronograma e riscos do MVP 2) finalizada e enviada ao professor até 27/08, 23h59.
Fase 1 — Infraestrutura e modelagem	Miguel Angelo Dufloth Filho	18/08 a 27/08 (em paralelo à Fase 0)	Pacote <code>apps/api</code> criado; <code>docker-compose</code> com PostgreSQL local funcionando; schema modelado e primeira <i>migration</i> (Alembic) aplicada a partir do modelo ER. Esta é a entrega técnica concreta do grupo até o dia 27/08.
Fase 2 — Identidade e cadastros base	Miguel Angel Balladares Huertas	28/08 a 10/09	RF06 (login, JWT, RBAC) e RF01/RF02 (CRUD de clientes e veículos) implementados, testados manualmente via Swagger.
Fase 3 — Núcleo operacional	Leonardo Lotério de Lima	11/09 a 24/09	RF05 (catálogo) e RF03/RF04 (emissão de OS e histórico de status) implementados, com cálculo de orçamento a partir do catálogo.

(continua na próxima página)

(continuação)

Fase	Responsável	Período	Entrega da fase
Fase 4 — Integração front-end e portal do cliente	Lucas Honorato dos Santos	25/09 a 08/10	Telas de login, gestão de OS, cadastro de clientes/veículos e portal do cliente consumindo a API real; suíte de testes automatizados das regras críticas de RF03/RF04.
Fase 5 — CI/CD e deploy de validação	Miguel Angelo Du-floth Filho e Lucas Honorato dos Santos	09/10 a 15/10	Pipeline de GitHub Actions (lint + <code>pytest</code>) ativo a cada <i>pull request</i> ; deploy de validação em Render (API/BD) e Vercel (front-end).

Observação: estas datas são o planejamento da equipe a partir de hoje; ficam sujeitas a ajuste caso o professor orientador do PAC VI sinalize um calendário oficial diferente para os marcos do MVP 2.

2.7 Acompanhamento e comunicação

Para reduzir o risco de atrasos silenciosos — já identificado na Seção 2.8 como sobreposição de calendário com outras disciplinas — a equipe adota:

- **Reunião semanal de status**, toda sexta-feira, com atualização do progresso de cada fase e realocação de tarefas entre os quatro integrantes caso alguma fase esteja atrasada.
- **Quadro de acompanhamento no GitHub** (Issues/Projects), com uma *issue* por requisito (RF01–RF06) e por entregável técnico da Seção 2.4, atribuída ao responsável da fase correspondente na Tabela 2.
- Cada responsável atualiza o status da própria *issue* até a véspera da reunião semanal, permitindo identificar bloqueios antes que afetem a fase seguinte.

2.8 Riscos específicos do MVP 2

Risco	Categoria	Prob.	Impacto	Nível	Mitigação
Inconsistências de contrato entre API real e front-end mockado	Técnico	Média	Médio	Médio	Contrato definido via OpenAPI/Swagger já na Fase 1; front-end consome o schema gerado automaticamente em vez de assumir formatos manualmente.

(continua na próxima página)

(continuação)

Risco	Categoria	Prob.	Impacto	Nível	Mitigação
Curva de aprendizado da equipe com FastAPI, PostgreSQL e Alembic	Organizacional	Média	Médio	Médio	Sessão interna de <i>pair programming</i> no início da Fase 1, com tutorial guiado compartilhado entre os quatro integrantes antes do início da Fase 2.
Sobreposição de calendário com entregas de outra disciplina do semestre	Organizacional	Média	Baixo	Baixo	Reuniões semanais fixas (Seção 2.7) para identificar atrasos cedo e redistribuir tarefas entre os quatro integrantes quando necessário.
Atraso na configuração de infraestrutura de deploy (Render/Vercel)	Técnico	Baixa	Médio	Baixo	Fase 5 isolada como etapa final sem dependências pendentes de outras fases; o deploy de validação pode escorregar sem travar as entregas anteriores do MVP 2.
Riscos de segurança já mapeados no PAC V (SQL Injection, falhas de autenticação) migrarem do papel para a implementação sem a devida mitigação	Segurança	Baixa	Alto	Médio	Checklist de segurança (Bcrypt, JWT, ORM, HTTPS) revisado como critério de aceite em cada <i>pull request</i> , não apenas ao final do MVP 2.